



151-0854-00L

Autonomous Mobile Robots

E05: Cameras

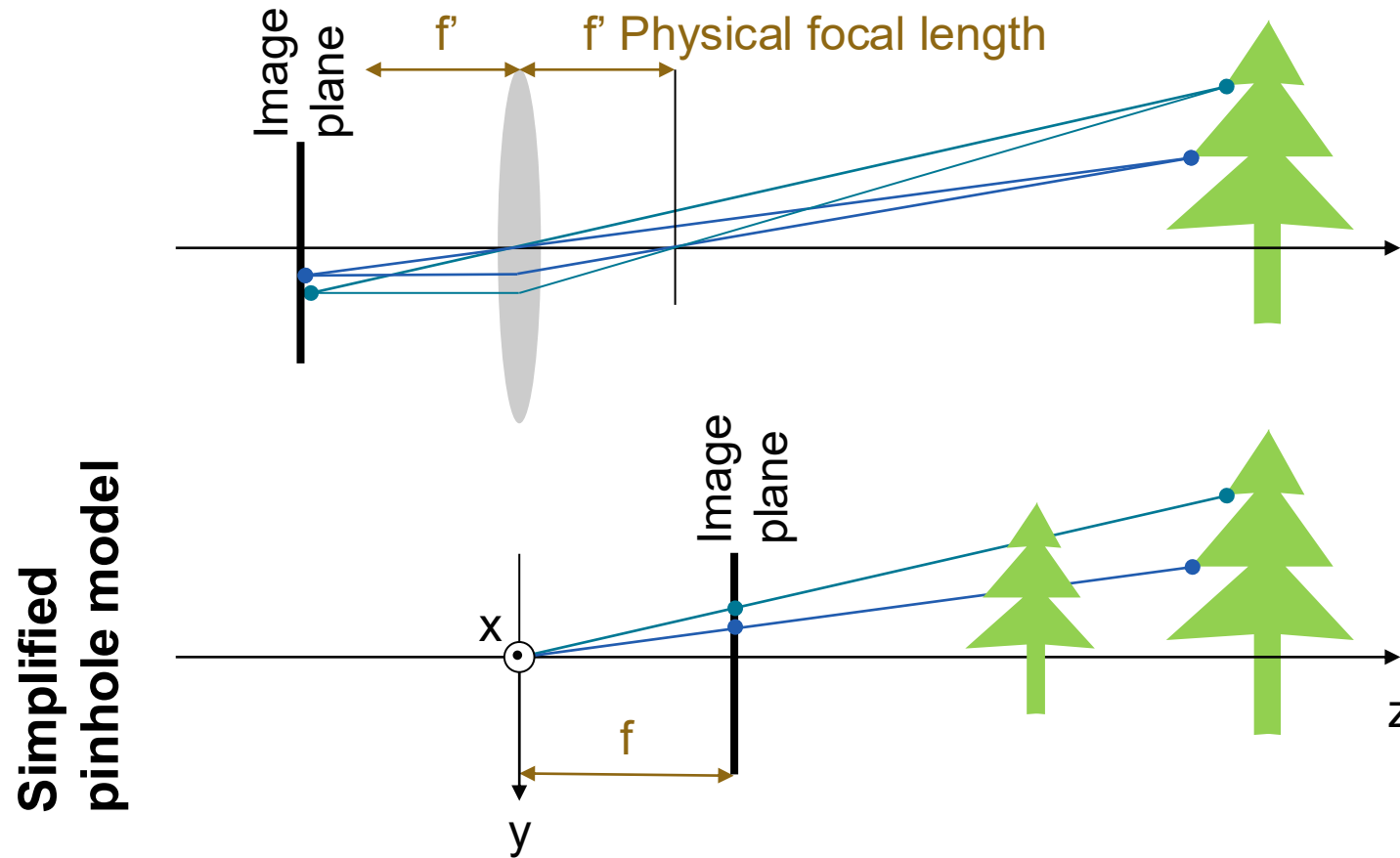
**Stefan Leutenegger, Tianyi Zhang, Cédric Le Gentil,
Simon Boche**



Overview

- Recap: Camera Model
 - Projection
 - Back-projection
- **Task 1: Camera Projection and Back-Projection**
 - Written Exercise
 - Python Implementation
 - Real-World Data Visualization
- Recap: Stereo Vision
 - Stereo Triangulation
 - Uncertainty Analysis
- **Task 2: Stereo Vision**
 - Written Exercise
 - Python Implementation
 - Real-World Data Visualization

The Camera



Pinhole projection:

$$\begin{bmatrix} u \\ v \end{bmatrix} = \frac{f}{z} \begin{bmatrix} x \\ y \end{bmatrix}$$

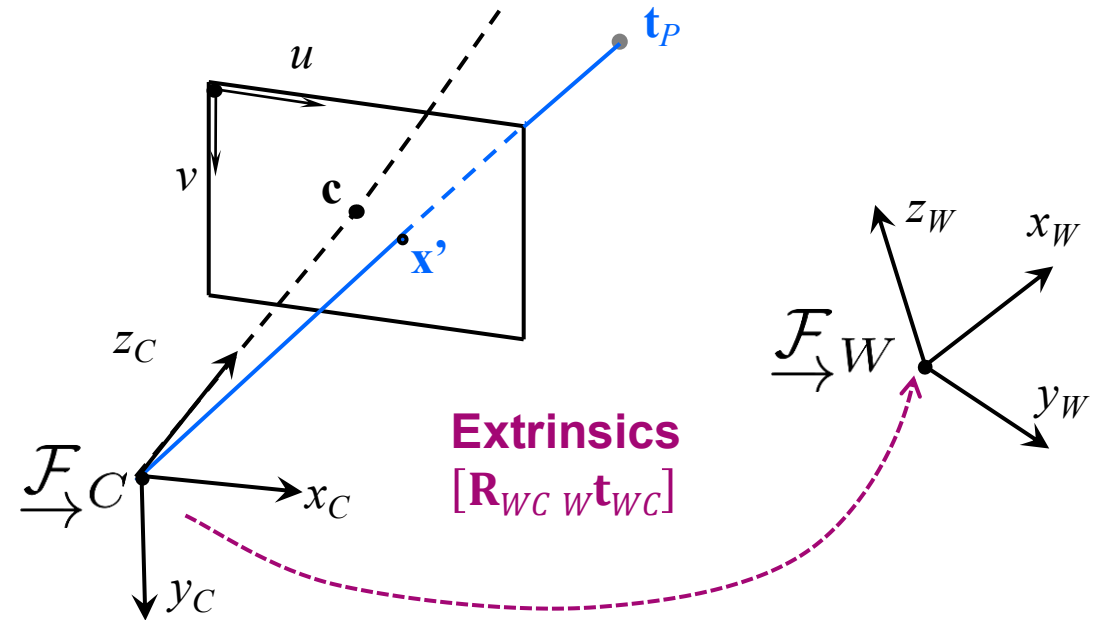
Perspective projection

- If we have the point to be projected represented in a different frame, e.g. $\mathcal{F}_W$, i.e. ${}_W\mathbf{t}_P$, we first have to transform:

$$\mathbf{u} = \begin{bmatrix} \lambda u \\ \lambda v \\ \lambda \end{bmatrix} = \mathbf{K}_C \mathbf{t}_P = \mathbf{K} [\mathbf{R}_{CW} \ \mathbf{c}_{CW}] {}_W\mathbf{t}_P.$$

Perspective Projection Matrix

In homogeneous coordinates



Perspective projection

1. We project the point ${}_c\mathbf{t}_P = [t_x, t_y, t_z]^T$ onto the image plane at distance $t_z = 1$:

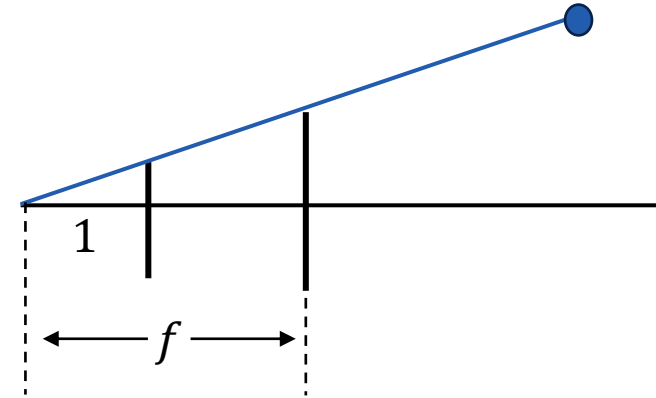
$$\mathbf{x}' = [x', y']^T = \left[\frac{t_x}{t_z}, \frac{t_y}{t_z} \right]^T.$$

2. To convert the projected point $\mathbf{x}' = [x', y']^T$, from unit-free image plane coordinates to the pixel coordinates $\mathbf{u} = [u, v]^T$, we need to account for:
 - the pixel coordinates of the camera optical center $\mathbf{c} = [c_u, c_v]$ in [pixels],
 - scale factor f in [pixels] for the pixel-size.

Thus, we obtain:

$$u = c_u + f x' = c_u + f \frac{t_x}{t_z},$$

$$v = c_v + f y' = c_v + f \frac{t_y}{t_z}.$$



Perspective projection | From scene points to pixels

1. We project the point ${}_c\mathbf{t}_P = [t_x, t_y, t_z]^T$ onto the image plane at distance $t_z = 1$:

$$\mathbf{x}' = [x', y']^T = \left[\frac{t_x}{t_z}, \frac{t_y}{t_z} \right]^T.$$

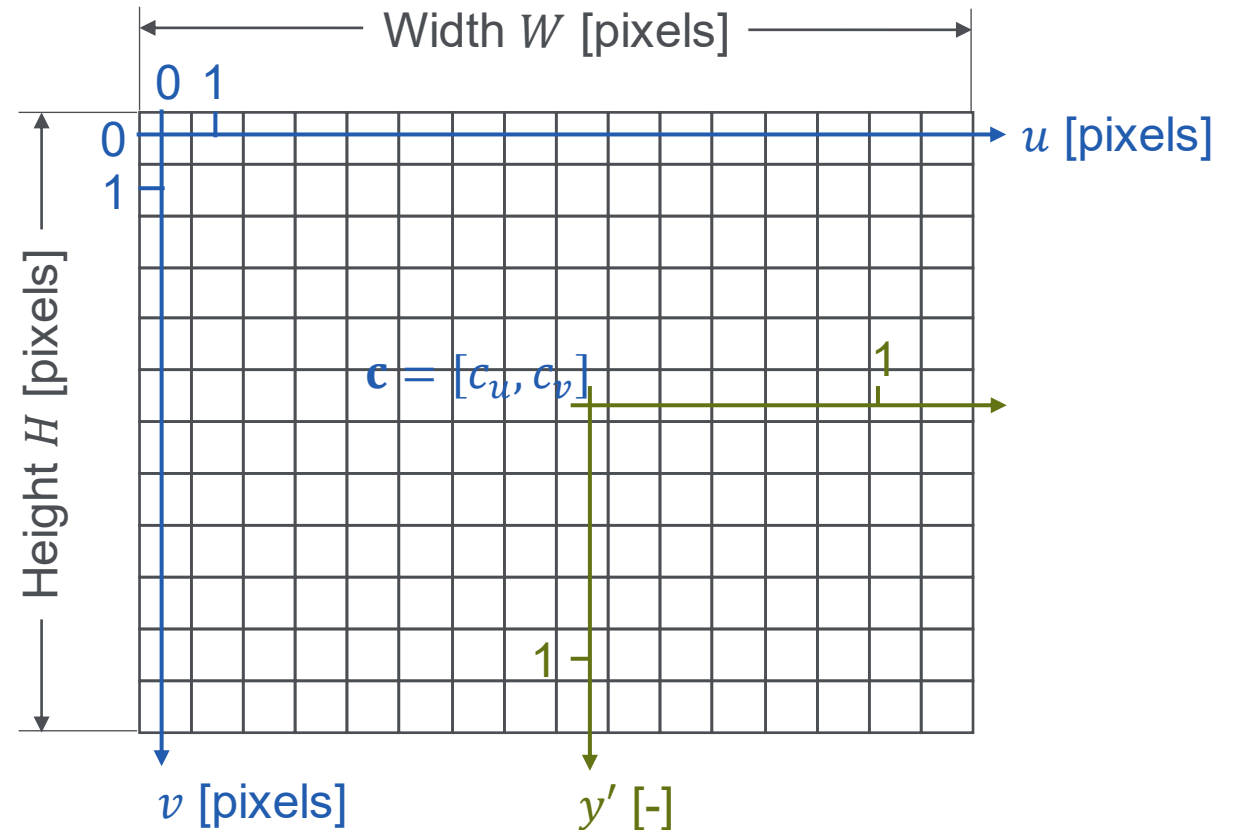
2. To convert the projected point $\mathbf{x}' = [x', y']^T$, from unit-free image plane coordinates to the pixel coordinates $\mathbf{u} = [u, v]^T$, we need to account for:

- the pixel coordinates of the camera optical center $\mathbf{c} = [c_u, c_v]$ in [pixels],
- scale factor f in [pixels] for the pixel-size.

Thus, we obtain:

$$u = c_u + f x' = c_u + f \frac{t_x}{t_z},$$

$$v = c_v + f y' = c_v + f \frac{t_y}{t_z}.$$



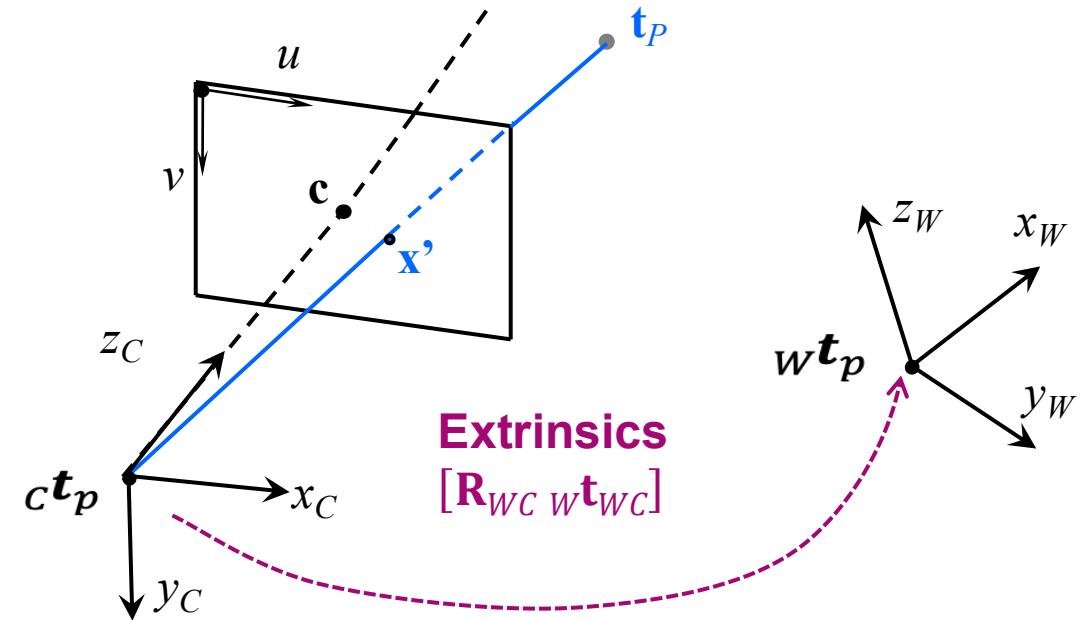
Perspective projection | From scene points to pixels

- If we have the point to be projected represented in a different frame, e.g. ${}^w\mathbf{t}_p$, i.e. ${}^w\mathbf{t}_P$, we first have to transform:

$$\mathbf{u} = \begin{bmatrix} \lambda u \\ \lambda v \\ \lambda \end{bmatrix} = \mathbf{K} {}^c\mathbf{t}_p = \mathbf{K} [\mathbf{R}_{cw} \mid {}^c\mathbf{t}_{cw}] {}^w\mathbf{t}_p.$$

Perspective Projection Matrix

In homogeneous coordinates



$${}^w\mathbf{t}_p \xrightarrow{[\mathbf{R}_{cw} \mid {}^c\mathbf{t}_{cw}]} {}^c\mathbf{t}_p \xrightarrow[p(\cdot)]{\begin{bmatrix} t_x & t_y \\ t_z & t_z \end{bmatrix}} \mathbf{x}' \xrightarrow[k(\cdot)]{\begin{bmatrix} f_u x'_u + c_u \\ f_v x'_v + c_v \end{bmatrix}} \mathbf{u}$$

$$u = k(p({}^c\mathbf{t}_p))$$

Pinhole Camera Projection with Distortion | From scene points to pixels

Complete Model: $\mathbf{u} = \mathbf{k}(\mathbf{d}(\mathbf{p}(\mathbf{C}\mathbf{t}_P)))$

1. Project point to unit plane:

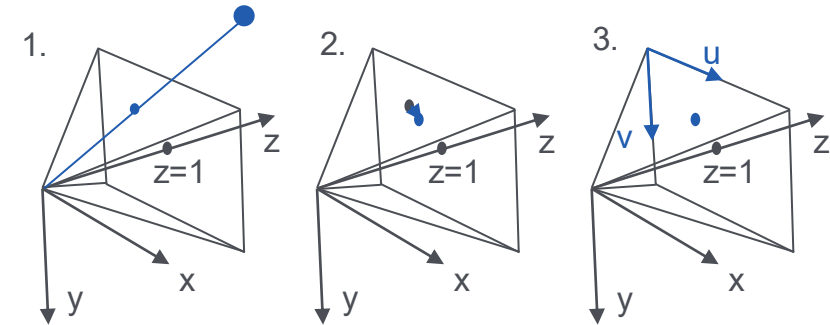
$$\mathbf{x}' = \begin{bmatrix} x' \\ y' \end{bmatrix} = \mathbf{p} \left(\begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \right) = \frac{1}{t_z} \begin{bmatrix} t_x \\ t_y \end{bmatrix}.$$

2. Apply distortion model – e.g. Radial-Tangential (with $r^2 = x'^2 + y'^2$):

$$\mathbf{x}'' = \mathbf{d}(\mathbf{x}') = \frac{1+k_1r^2+k_2r^4+k_3r^6}{1+k_4r^2+k_5r^4+k_6r^6} \begin{bmatrix} x' \\ y' \end{bmatrix} + \begin{bmatrix} 2p_1x'y' + p_2(r^2 + 2x'^2) \\ p_1(r^2 + 2y'^2) + 2p_2x'y' \end{bmatrix}.$$

3. Scale and centre:

$$\mathbf{u} = \mathbf{k}(\mathbf{x}'') = \begin{bmatrix} f_u & 0 \\ 0 & f_v \end{bmatrix} \mathbf{x}'' + \begin{bmatrix} c_u \\ c_v \end{bmatrix}.$$



¹⁾ Following OpenCV, http://docs.opencv.org/modules/calib3d/doc/camera_calibration_and_3d_reconstruction.html.

k_1, k_2 : Radial distortion parameters.

f_u, f_v : x/y focal lengths in pixels.

k_3-k_6 : (Optional) radial distortion parameters.

c_u, c_v : Image centre in pixels.

p_1, p_2 : Tangential distortion parameters.

Pinhole Camera Projection with Distortion | Back-projection: from pixels to rays

Now we want to invert the model...

Note: we can only find a ray, not the actual 3D point!

1. Change to unit plane coordinates:

$$\mathbf{x}'' = \mathbf{k}^{-1}(\mathbf{u}) = \begin{bmatrix} \frac{1}{f_u} & 0 \\ 0 & \frac{1}{f_v} \end{bmatrix} \left(\mathbf{u} - \begin{bmatrix} c_u \\ c_v \end{bmatrix} \right)$$

2. Un-distort (e.g. Radial-Tangential):

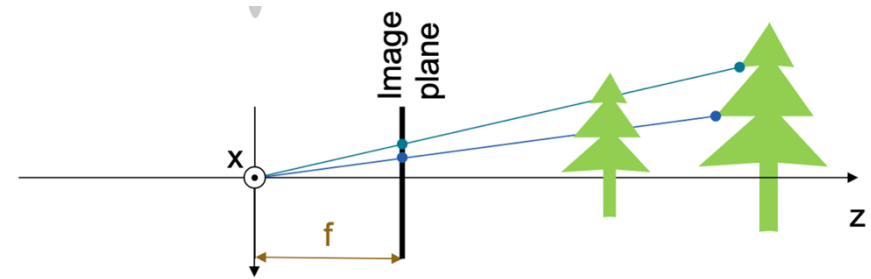
$$\mathbf{x}' = \mathbf{d}^{-1}(\mathbf{x}'').$$

Note: this generally has to be done numerically (e.g. using gradient descent or Gauss-Newton)

3. Compute the ray – simply remember we are operating in the unit plane...

$${}^C\mathbf{r} = \begin{bmatrix} \mathbf{x}' \\ 1 \end{bmatrix}.$$

Deep Learning-Based Depth Estimation



Monocular depth prediction:

- Input: single RGB image
- Output: per-pixel depth (even metric!)
- Training: direct supervision (e.g. RGB-D camera) and/or self-supervision image warp consistency using sequences with known poses



Example results in [1]. Figure taken from [1].

[1] Ranftl, René, et al. "Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer." arXiv preprint arXiv:1907.01341 (2019).

Task 1: Camera Projection and Back-Projection

You are supposed to implement the camera projection and back-projection functions of a pinhole camera model with radial-tangential distortion into the Python script `e05_pinhole_camera.py`.

For installation of required packages for this and the next task, we provide you with a `requirements.txt`. You can install them using e.g. `pip install -r requirements.txt`

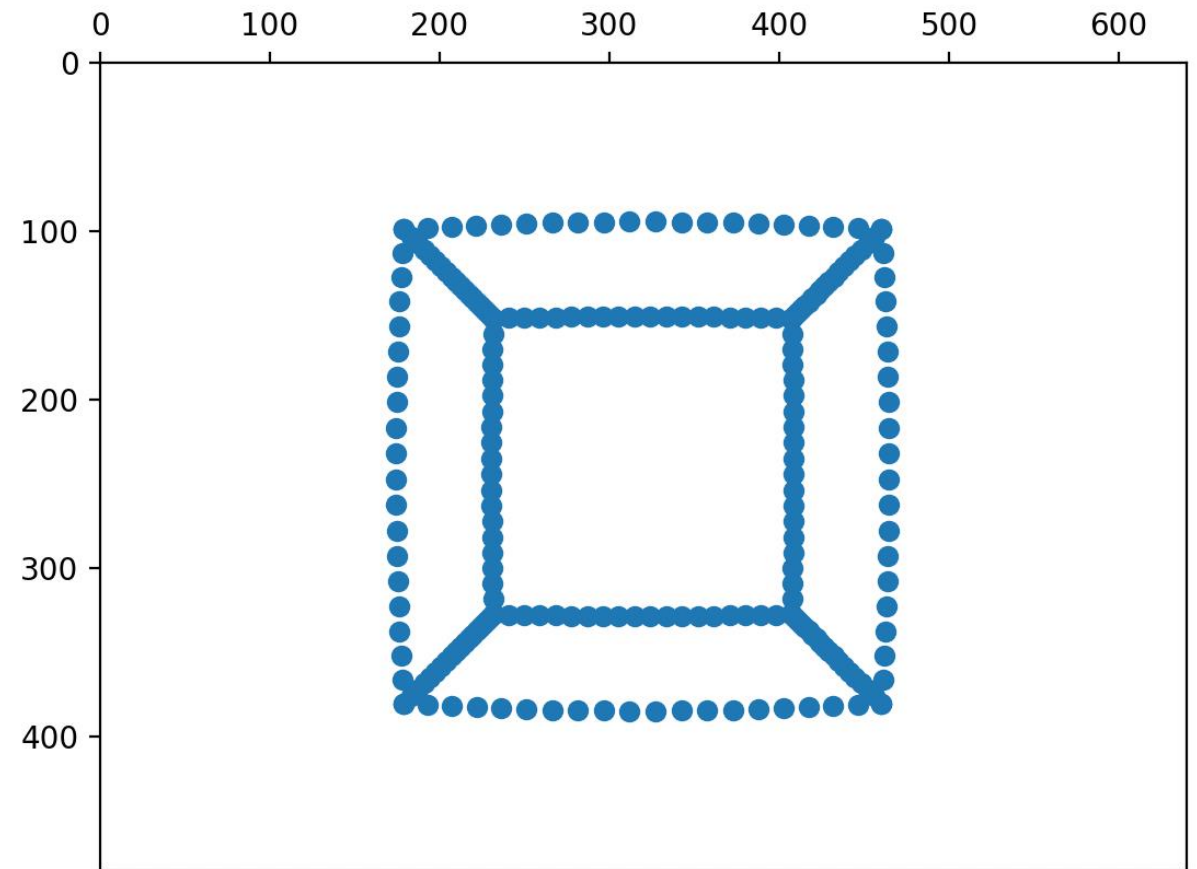
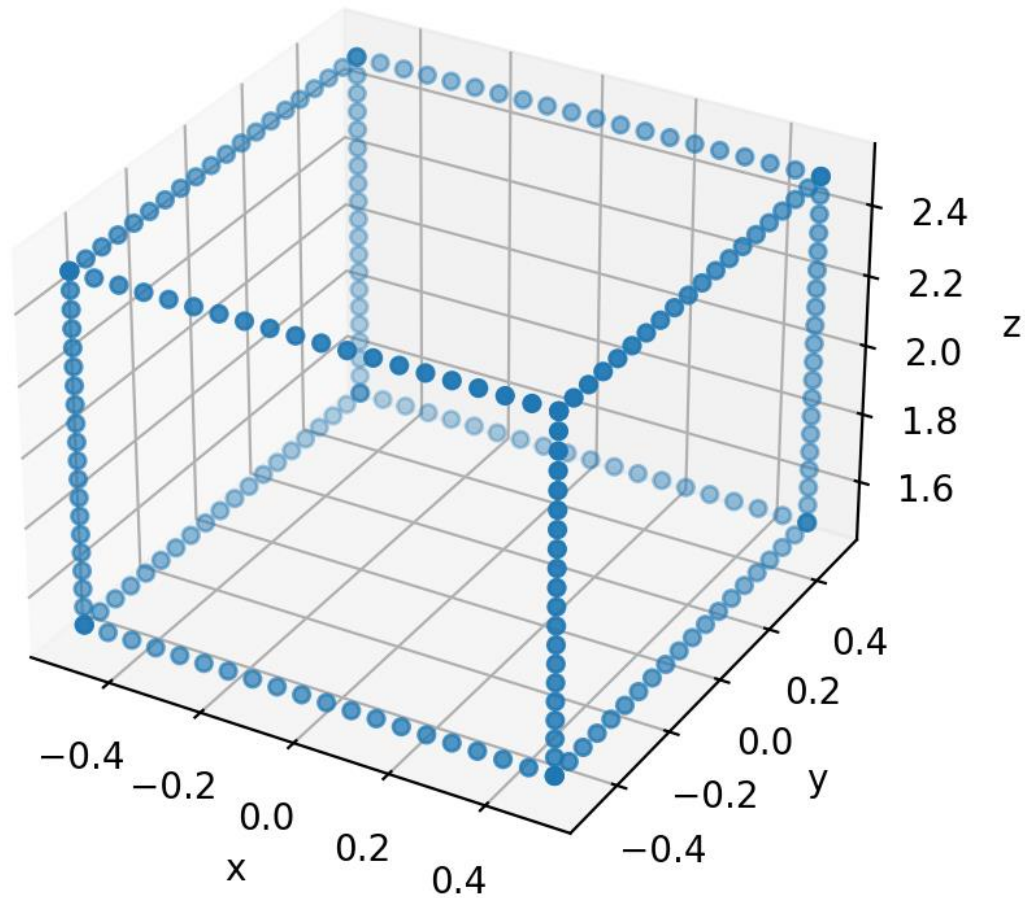
Task 1: Camera Projection and Back-Projection

- i) Before the Python implementation, let's first review the concepts with a simple camera projection and back-projection example. Consider a camera with intrinsic parameters $f_u = f_v = 100$, $c_u = 320$, $c_v = 240$, radial distortion parameters $k_1 = 0.2$, $k_2 = k_3 = k_4 = k_5 = k_6 = 0$, and tangential distortion parameters $p_1 = 0.01$ and $p_2 = 0.01$. A 3D point in camera coordinates is ${}_C\mathbf{t}_P = [1, 1, 2]^T$. Compute 2D projected point $\mathbf{u}$ in pixel coordinate.

Task 1: Camera Projection and Back-Projection

- ii) Study the code: note how the necessary parameters are part of the `PinholeCamera` class and how you are meant to split things into functions `p(.)`, `d(.)`, and `k(.)` as introduced in the lecture. Also note the use of the `RadialTangentialDistortion` helper class, which could now be easily swapped for a different distortion model.
- iii) Implement the `project(.)` function. Make sure to return an empty array, if the projection is invalid. Also complete the cell that renders a cube in order to check qualitatively if things are correct.
- iv) Implement the `backProject(.)` function. Note the readily available `d_inverse(.)` function (since it requires a numeric solver, we implemented this for you, but be aware that it will use your implementation of `distort(.)` in the `RadialTangentialDistortion` class to do so).
- v) To really make sure you did everything correctly, implement a unit test that generates a random visible 3D point from a random image coordinate using the `backProject(.)` function (but also at a random depth), and then projects it into the image using `project(.)` – where the projection should then match the originally created image point.

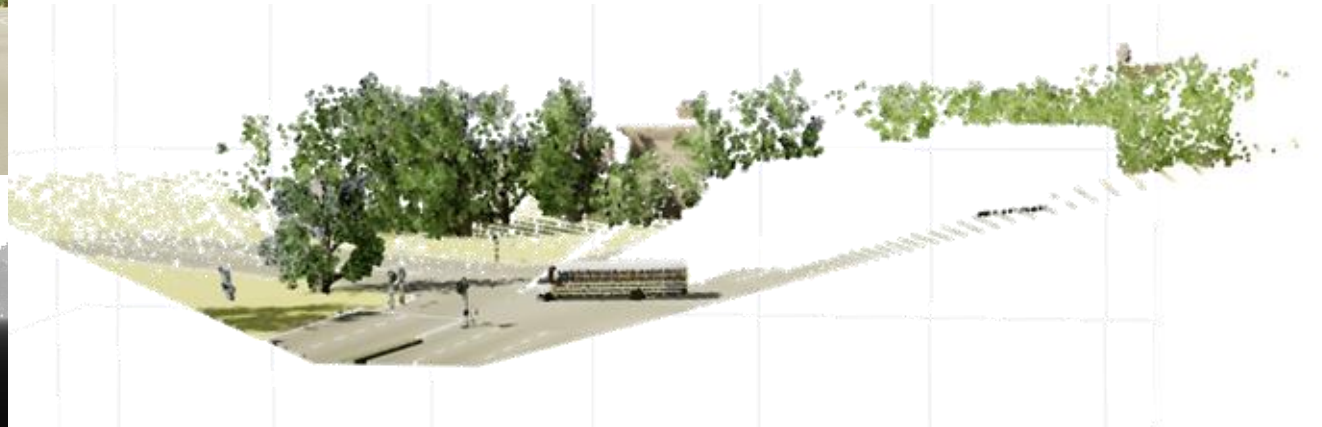
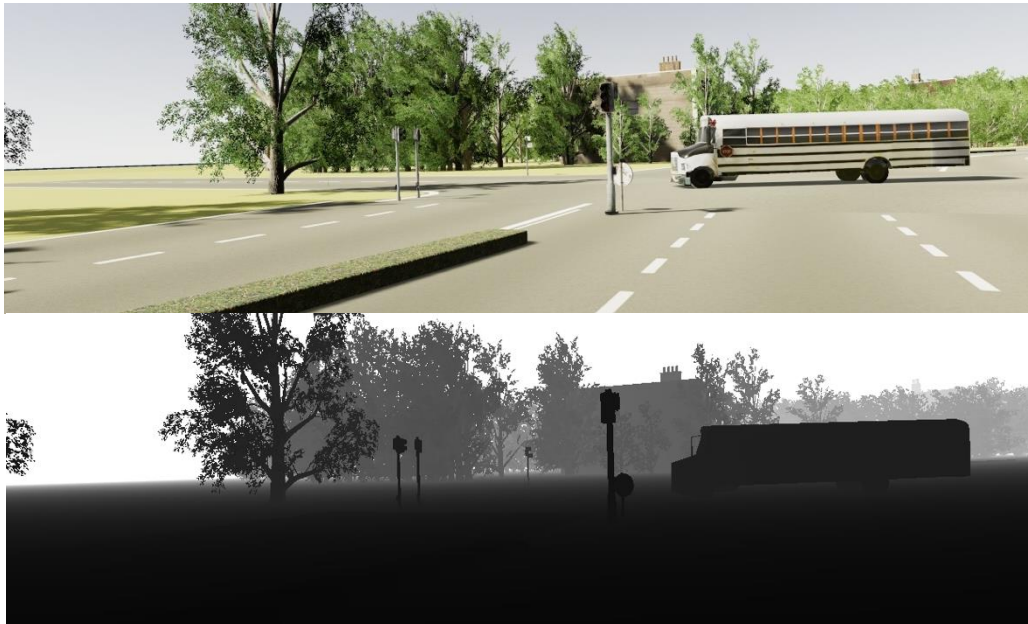
Task 1: Camera Projection and Back-Projection



Task 1: Camera Projection and Back-Projection

- vi) Visualize and compare the real data. We provide ground-truth depth maps and depth maps estimated by state-of-the-art monocular network Metric3Dv2¹. Qualitatively compare them by using the above `PinholeCamera` class to back-project the images into 3D with the given depth maps and visualize the 3D point clouds. Then quantitatively compare them by computing the average absolute relative depth error

$$\delta_{rel} = |d_i^{est} - d_i^{gt}| / d_i^{gt} \quad (1)$$



Task 1: Camera Projection and Back-Projection

- vi) Visualize and compare the real data. We provide ground-truth depth maps and depth maps estimated by state-of-the-art monocular network Metric3Dv2^[1]. Qualitatively compare them by using the above `PinholeCamera` class to back-project the images into 3D with the given depth maps and visualize the 3D point clouds. Then quantitatively compare them by computing the average absolute relative depth error

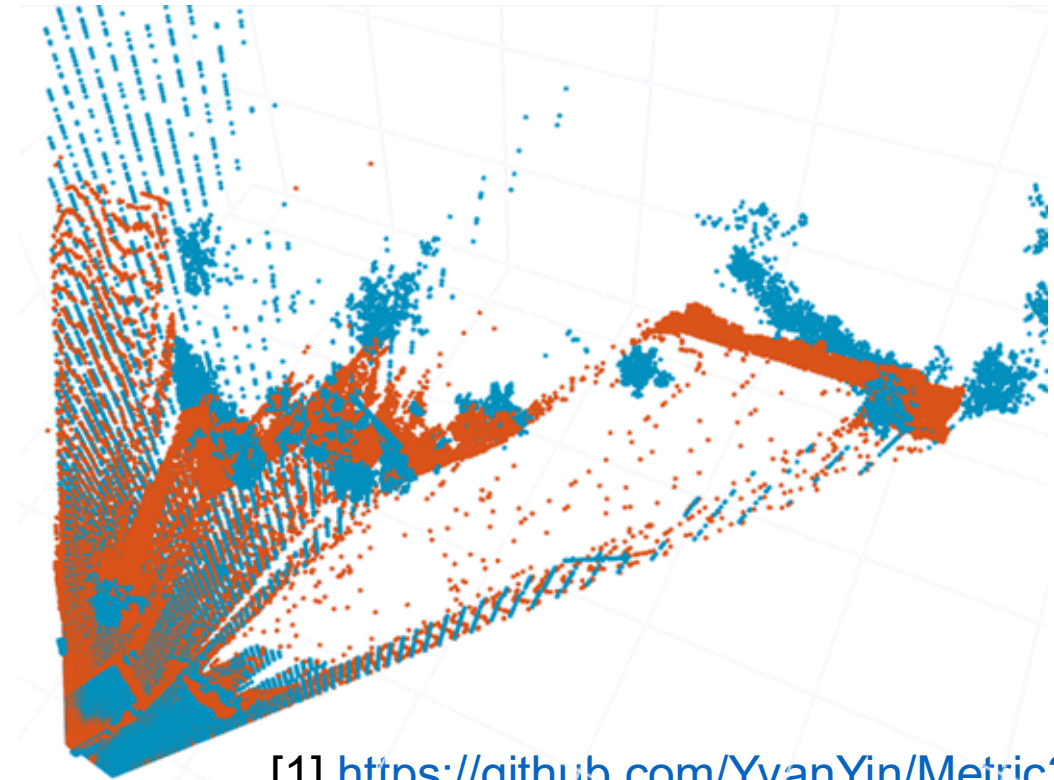
$$\delta_{rel} = |d_i^{est} - d_i^{gt}| / d_i^{gt} \quad (1)$$



ground-truth depth



monocular depth from Metric3Dv2[1]



[1] <https://github.com/YvanYin/Metric3D>

Dense Depth Estimation: Classic Stereo

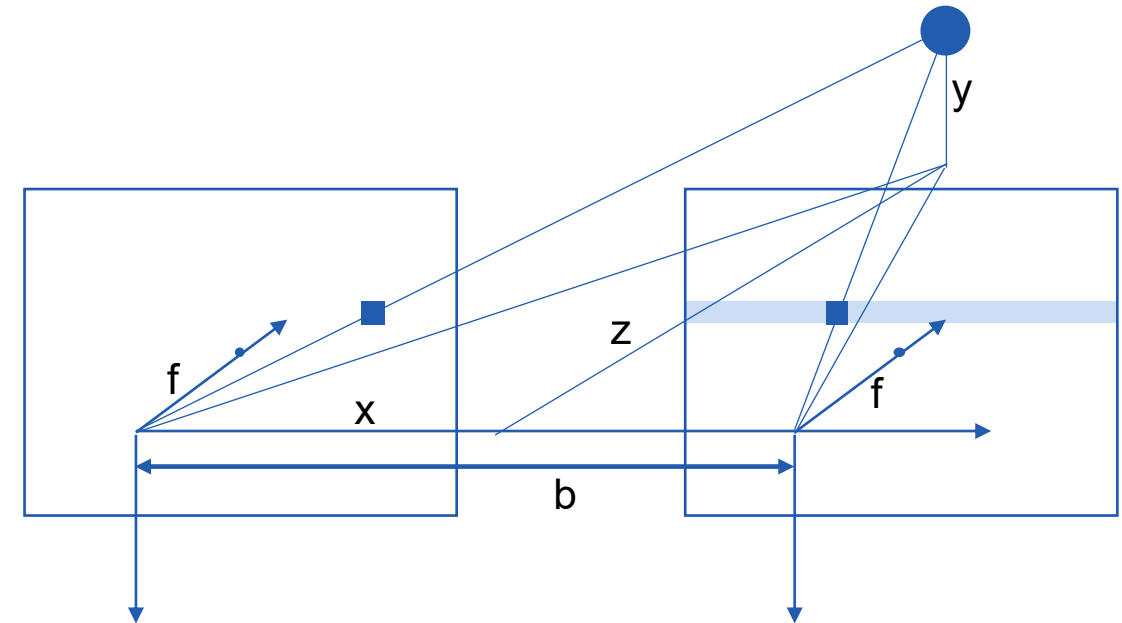
Consider a **stereo-rectified image pair**

- Images in same plane
- Same focal lengths
- Same image centre
- Common camera x axis

Given: corresponding pixels $[u_l, v]$ and $[u_r, v]$

Find: depth d of the respective 3D point.

- Project point into left camera: $u_l = f \frac{x}{z} + cu$.
- Project point into right camera: $u_r = f \frac{x-b}{z} + cu$,
- Introducing the disparity $d = u_r - u_l$, we obtain $z = \frac{bf}{d}$.



Note: the art is how to find the correspondences (u_r to a selected u_l). Use e.g. Normalised Cross Correlation (NCC) of patches around u_r and u_l .

Dense Depth Estimation: Classic Stereo

Consider a **stereo-rectified image pair**

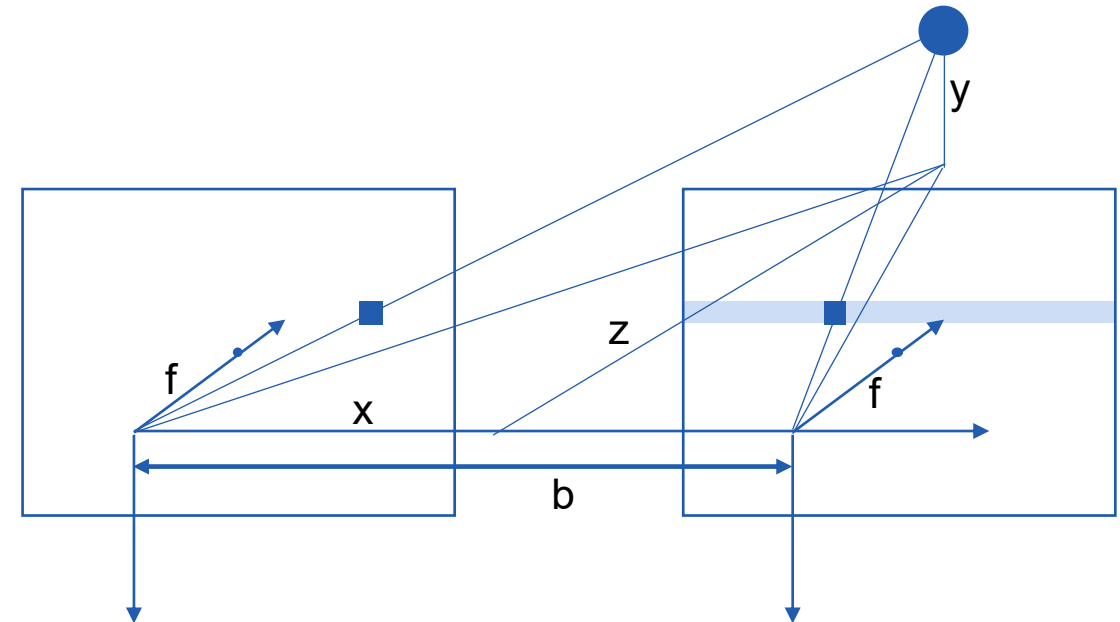
- Images in same plane
- Same focal lengths
- Same image centre
- Common camera x axis

Given: corresponding pixels $[u_l, v]$ and $[u_r, v]$

Find: depth d of the respective 3D point.

- Project point into left camera: $u_l = f \frac{x}{z} + cu$.
- Project point into right camera: $u_r = f \frac{x-b}{z} + cu$,
- Introducing the disparity $d = u_r - u_l$, we obtain $z = \frac{bf}{d}$.

Note: the art is how to find the correspondences (u_r to a selected u_l). Use e.g. Normalised Cross Correlation (NCC) of patches around u_r and u_l .

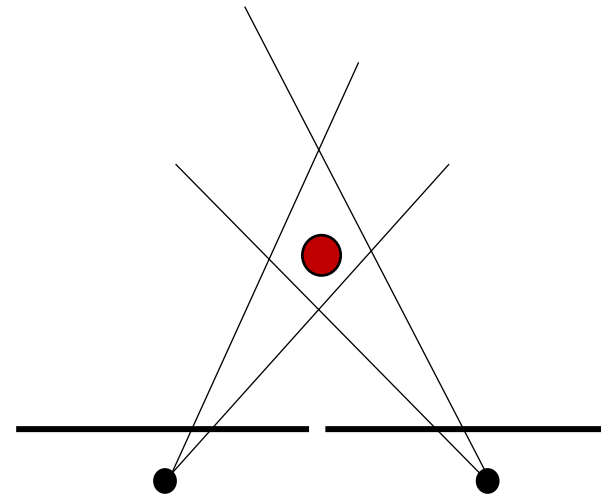
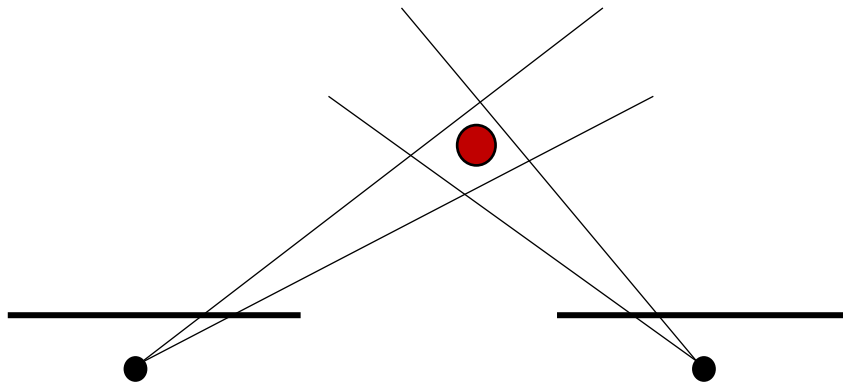


Uncertainty in Stereo Triangulation

- Correspondence / disparity often contain errors! How do these errors affect the accuracy of depth estimation?

$$z = \frac{bf}{d} \quad \text{— How camera parameters affect the the accuracy?}$$

- Hint: Qualitative analysis of uncertainty in stereo triangulation: How baseline affects accuracy

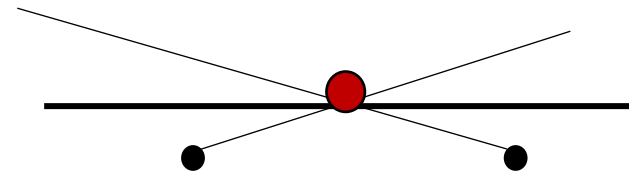
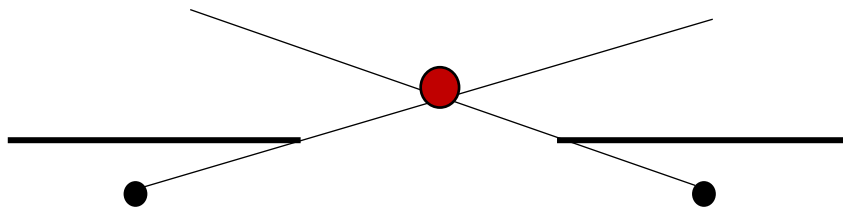


Uncertainty in Stereo Triangulation

- Correspondence / disparity often contain errors! How do these errors affect the accuracy of depth estimation?

$$z = \frac{bf}{d}$$

- Hint: Qualitative analysis of uncertainty in stereo triangulation



Task 2: Stereo Vision

This exercise considers the stereo triangulation problem. You will see how a stereo camera setup can be used to recover depth information from images in the Python script `e05_stereo_vision.py`. In this section, we assume rectified stereo camera setup without distortion.

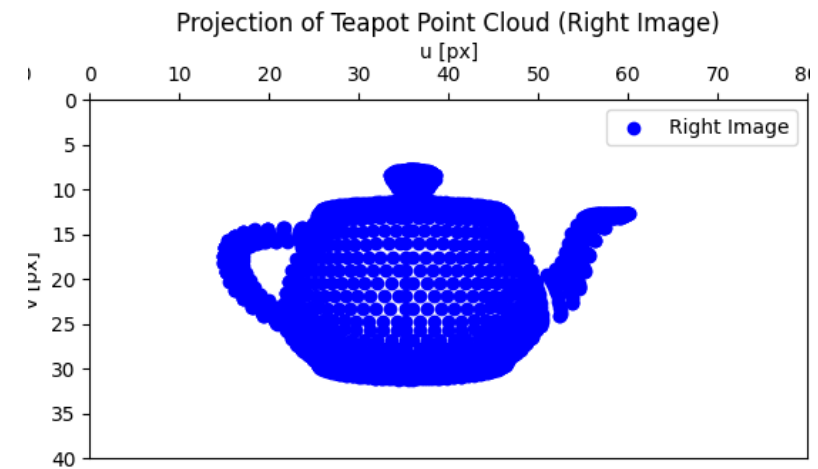
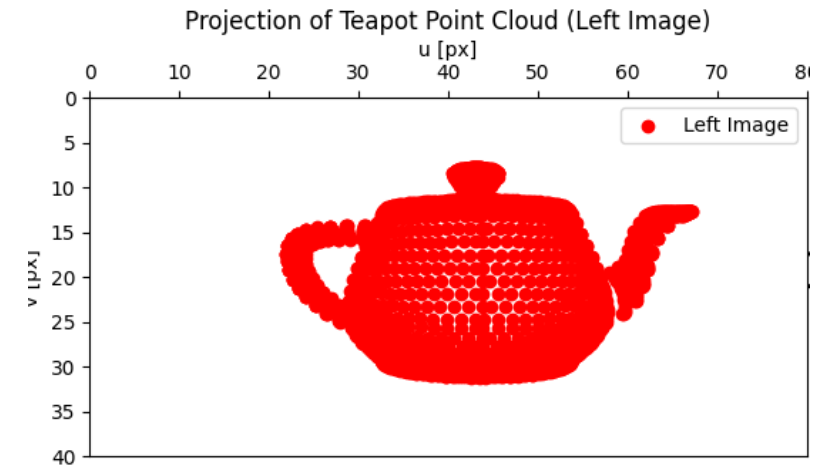
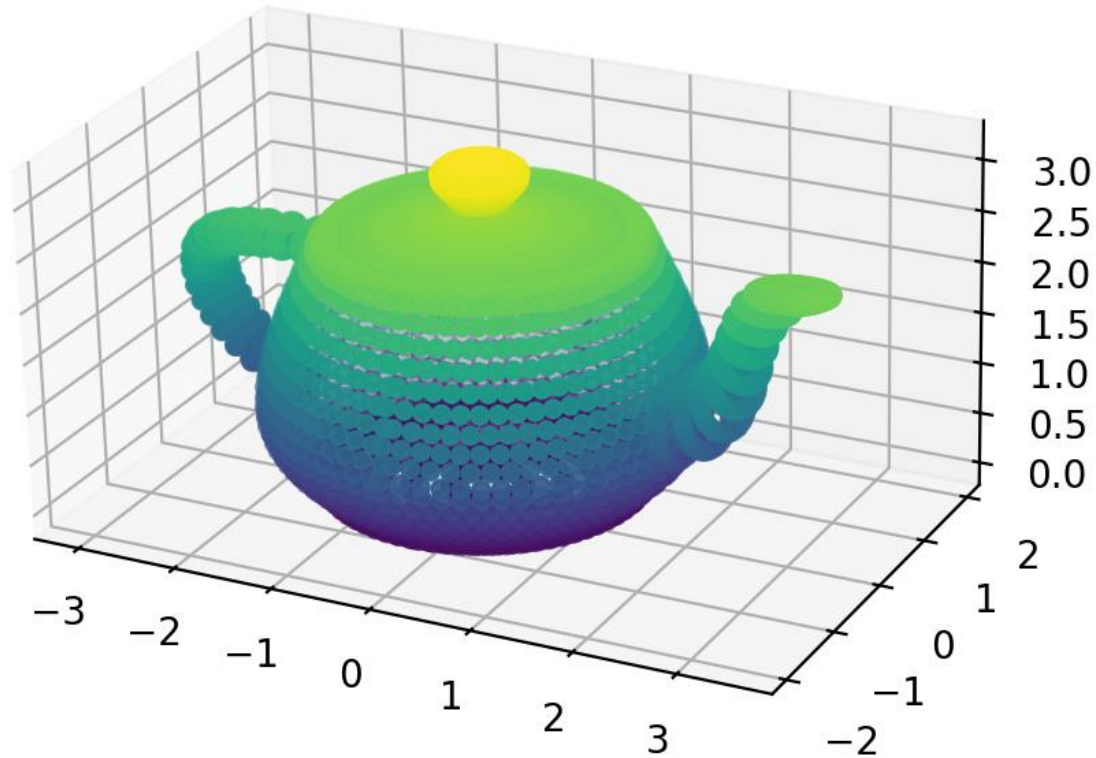
Task 2: Stereo Vision

- i) Before the Python implementation of the following tasks, let's first work through a written example. Consider rectified stereo cameras with focal length $f = 800$, principal point $(c_u, c_v) = (320, 240)$, and baseline $b = 0.2$ m.
- A 3D point is observed at pixel $u_l = 450$ in the left image and $u_r = 370$ in the right image. The vertical image coordinate of the point is $v = 300$. Compute the disparity d , depth z , and determine the 3D coordinates ${}_C \mathbf{t}_P$ in the left camera coordinate.
 - Consider that the disparity measurement is affected by noise of $\Delta d = 1$ pixel. Derive the depth uncertainty Δz as a function of depth. Then use this expression to compute the depth uncertainty for the above setting. Next, change the baseline b to 0.1 m and 0.4 m and recompute the depth uncertainty. Briefly explain how the accuracy of stereo depth estimation can be improved.

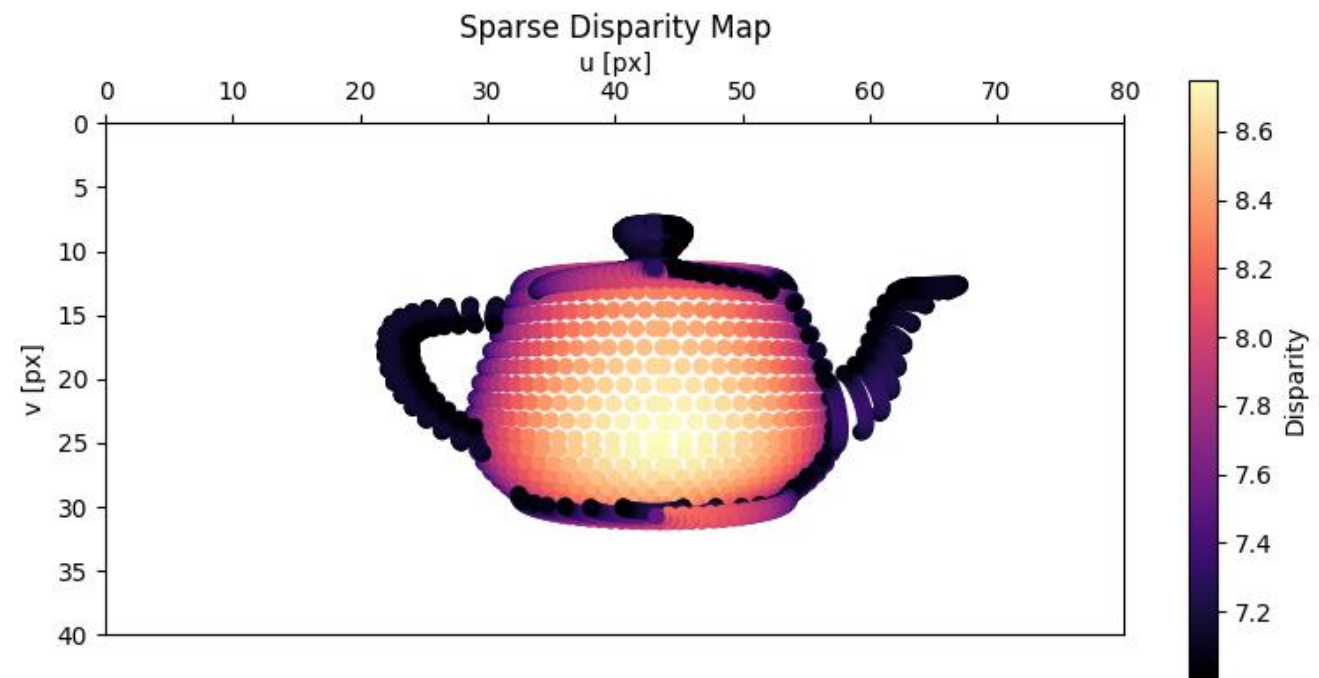
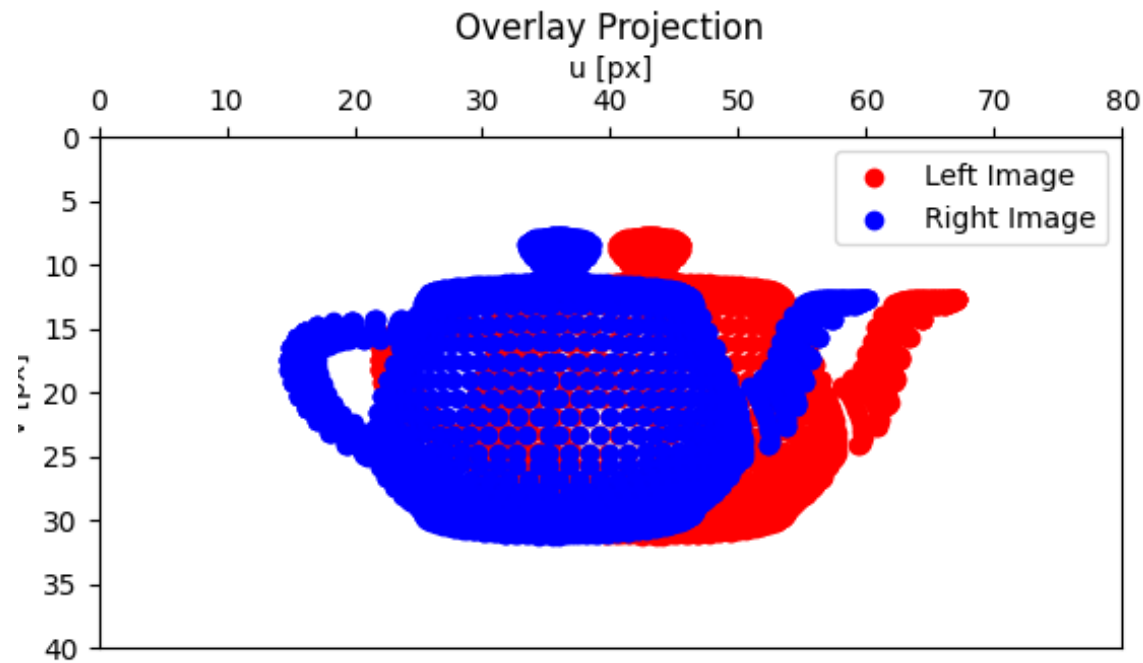
Task 2: Stereo Vision

- ii) Stereo projection. We provide two cameras in a given stereo setup together with the Utah teapot point cloud. Your first task is to project the 3D points into the two image planes, create the corresponding projection images, and visualize them. For simplicity, you may import the `PinholeCamera` class from Task 1. Note that you need to transform the points into the corresponding camera frame before projection.
- iii) Compute the disparity for all valid projected points. Visualize the disparity values as a color map using the readily available function `plotDisparity()`.
- iv) Reconstruct the 3D structure from the disparity map. Complete the function `stereoTriangulation()` and visualize the recovered 3D structure.
- v) Add white noise to the disparity map using the helper function `addNoise()`, visualize the triangulated 3D result, and study how the stereo baseline influences the reconstruction quality.

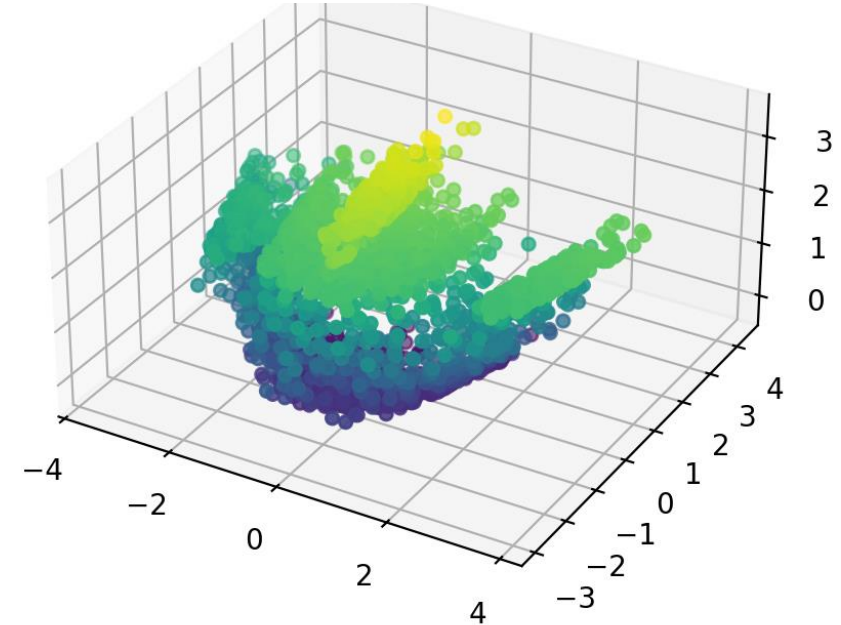
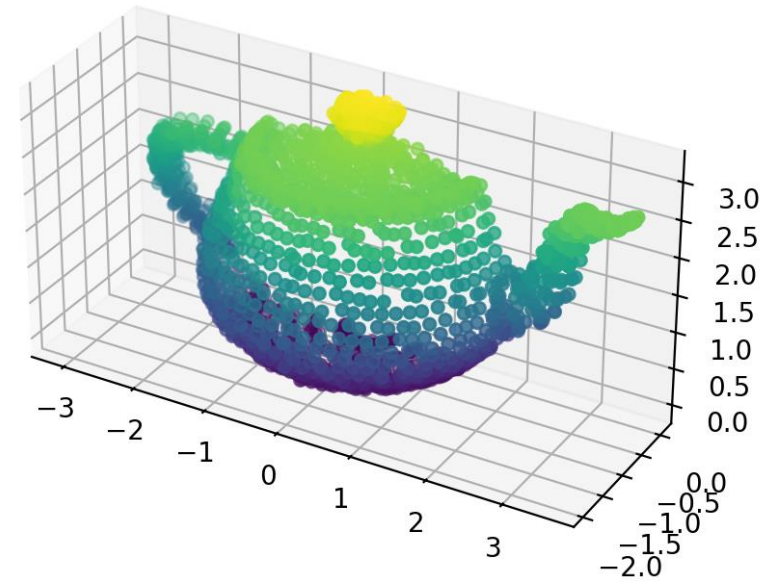
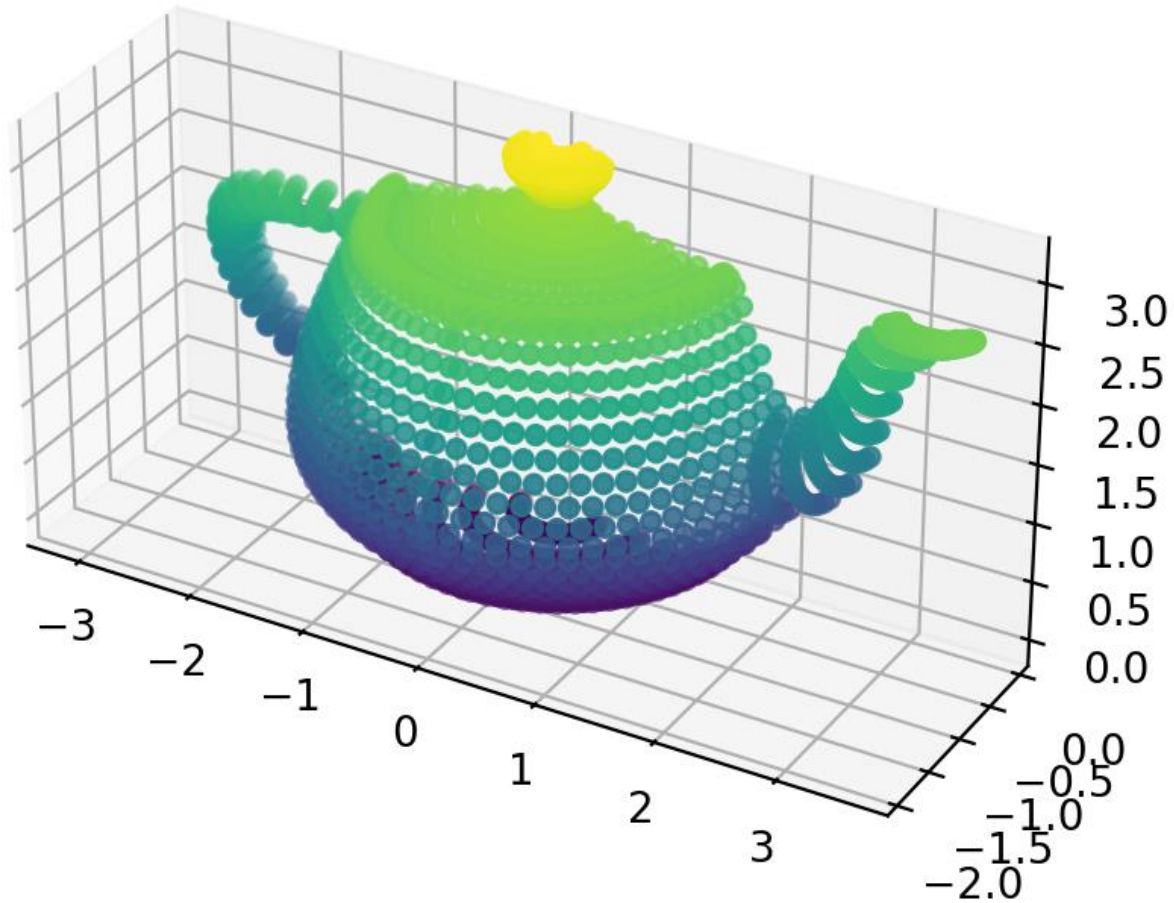
Task 2: Stereo Vision



Task 2: Stereo Vision

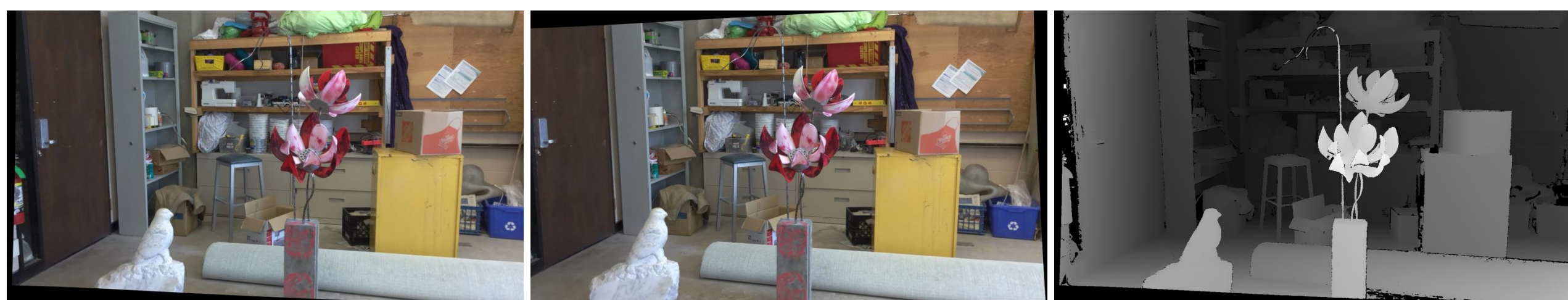


Task 2: Stereo Vision



Task 2: Stereo Vision

- vi) Triangulate the real-world data using the triangulation function above. We provide both the ground-truth disparity map and the disparity map estimated by FoundationStereo². Similar to Task 1, compute the average absolute relative depth error, visualize the point clouds, and compare them both qualitatively and quantitatively. You can find the data in the `./data/stereo/` folder

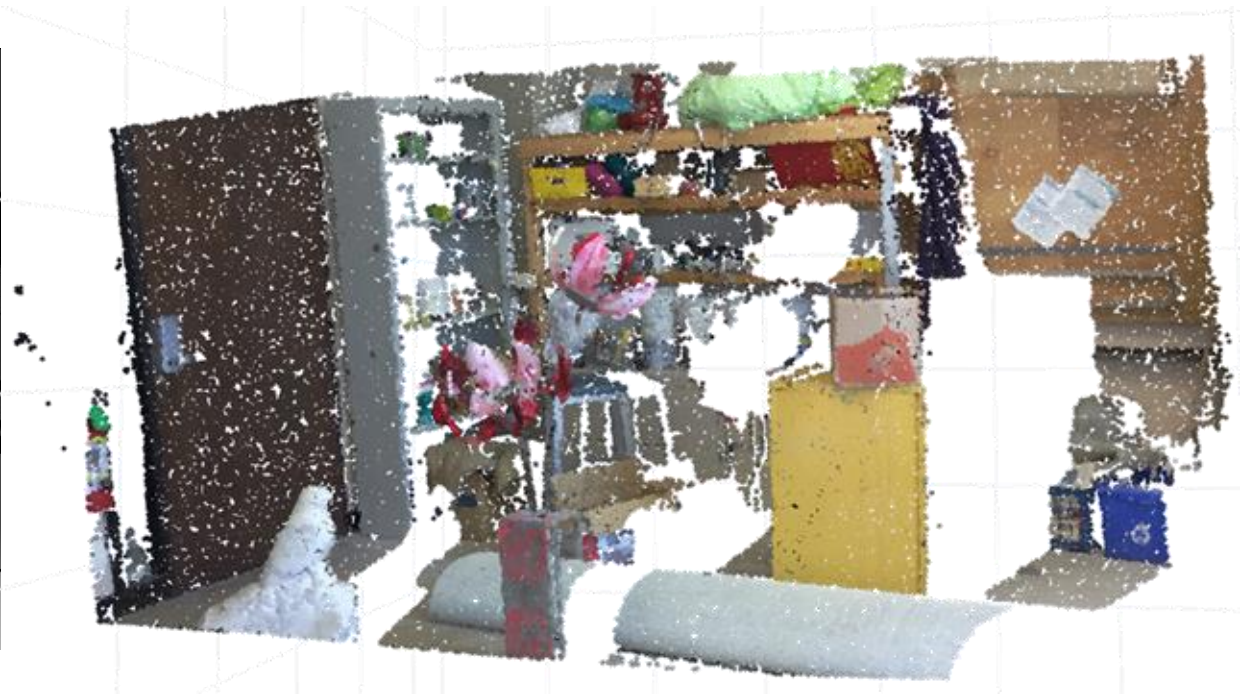


Task 2: Stereo Vision

- vi) Triangulate the real-world data using the triangulation function above. We provide both the ground-truth disparity map and the disparity map estimated by FoundationStereo². Similar to Task 1, compute the average absolute relative depth error, visualize the point clouds, and compare them both qualitatively and quantitatively. You can find the data in the `./data/stereo/` folder

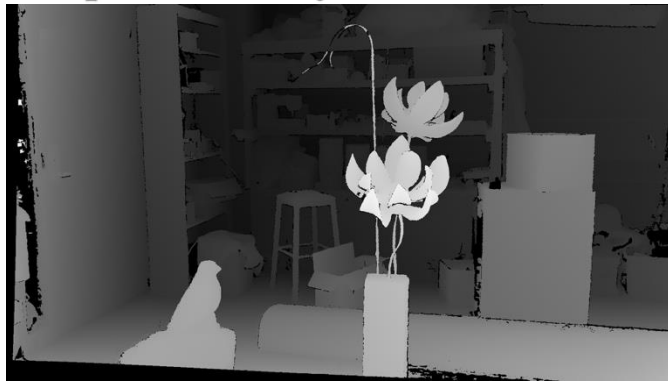


ground-truth disparity



Task 2: Stereo Vision

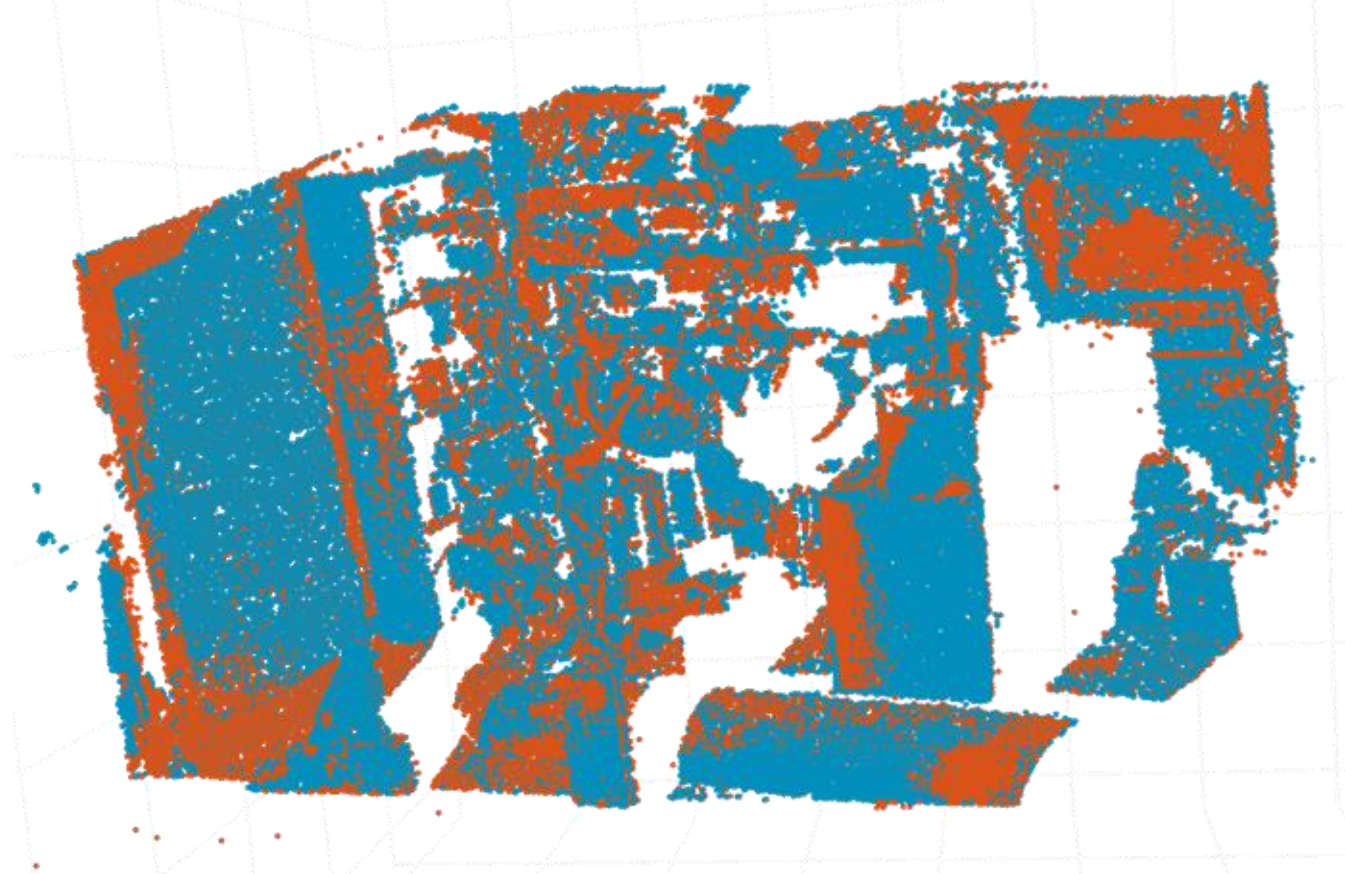
- vi) Triangulate the real-world data using the triangulation function above. We provide both the ground-truth disparity map and the disparity map estimated by FoundationStereo^[2]. Similar to Task 1, compute the average absolute relative depth error, visualize the point clouds, and compare them both qualitatively and quantitatively. You can find the data in the `./data/stereo/` folder



ground-truth disparity



disparity from FoundationStereo[2]



Thanks! Questions?